

Yaena Shin

Khaleed Solaiman

CMSC 421

September 20th, 2026

## Python Assignment #1

1.

### **KEY:**

Pseudocode for steps heuristic

Pseudocode for scrabble heuristic

Pseudocode for frequency heuristic

### Full Pseudocode for Function def h(self, node)

---

wordLength = length of string

mismatches = for every letter in state, if state[index] != goal[index], +1

if cost = "steps"

return mismatches

elif cost = "scrabble"

total = 0

for each i in range(wordlength)

if state[i] != goal[i]

```
needLetter = goal[i]
```

```
total += needLetter scrabble score
```

```
return total
```

```
elif cost = "frequency"
```

```
return mismatches
```

```
else
```

```
return error, invalid cost measure was entered
```

---

## Steps

For calculating the ‘steps’ cost, each change of letter is a cost of 1. Therefore, if you change 3 letters from state n to reach the goal word, the cost from n to goal state would be exactly 3. If you start at state n, you can estimate the heuristic by determining how many letters are different from the goal word’s letters at each corresponding index. This will always underestimate or be equal to the true cost. Because if there are exactly 2 letters that differ from state n to your goal word, the least amount of steps you can take is exactly 2. Assuming that changing both differing letters with needed letters will both be valid words, that is exactly 2 steps. However, some words may require more than 2 changes because changing one of the different letters to a goal letter corresponding to that position may be an invalid word. Therefore, using mismatched letters as a heuristic for steps cost will always be equal or less than to the true cost from state n.

## Scrabble

A similar idea to steps can be applied to the scrabble cost heuristic. For the pseudocode, I compared each index from the current word to the goal word, then took any differing letters of the goal word, found its scrabble letter value, then added them together to return as the heuristic value. Again, assuming that each differing letter in the current word can be replaced with a letter in the goal word that results in a valid word, the cost based on the scrabble dictionary will at minimum always be equal to the estimated heuristic or more if no valid words result from changing a differing letter with a needed letter. No matter what the current word is, the needed letters will always result in the same individual scrabble cost. If more letters need to be changed to reach valid words which then transition to the goal state, the cost will be higher than expected. Therefore, the demonstrated heuristic will always be equal or less than the true path cost.

### **Frequency**

The frequency cost heuristic is hard to determine due to its highly variable cost rate of each word. We are unable to generally assume the cheapest path for every single word because we cannot account for every one. Additionally, the given frequency values are highly irregular. However, we do know that all of the frequency values are positive, non-zero numbers and the minimum known number of moves to the goal word from state  $n$  with our variable mismatches. Since each replacement step adds some positive amount to the total cost, the true remaining cost must be greater than zero at least once for each mismatched position. This means the total remaining cost is guaranteed to be at least the number of mismatches, because you cannot complete the transformation using fewer than that many actions, and no value of frequencies are equal to 0. Even if every future step costs something like 0.000001, the true remaining cost is still a positive number that has to be added, and it cannot fall below the mismatch count.

2.

| <b>Word Pair</b>  | <b>Cost Measure</b> | <b>UCS Cost</b> | <b>UCS Nodes Expanded</b> | <b>A* Cost</b> | <b>A* Nodes Expanded</b> | <b>Reduction</b> |
|-------------------|---------------------|-----------------|---------------------------|----------------|--------------------------|------------------|
| <b>WHY, ASK</b>   | Steps               | 5               | 773                       | 5              | 16                       | 97.3%            |
| <b>WHY, ASK</b>   | Scrabble            | 9               | 765                       | 9              | 13                       | 98.3%            |
| <b>WHY, ASK</b>   | Frequency           | 30.64           | 764                       | 30.64          | 689                      | 9.82%            |
| <b>BEAR, DUCK</b> | Steps               | 4               | 691                       | 4              | 6                        | 99.1%            |
| <b>BEAR, DUCK</b> | Scrabble            | 11              | 2981                      | 11             | 6                        | 99.8%            |
| <b>BEAR, DUCK</b> | Frequency           | 34.24           | 1229                      | 34.24          | 795                      | 54.6%            |

- a. Based on my results, UCS and A\* always returned the same optimal cost.
- b. The largest reduction for A\* for the word pair WHY, ASK was the scrabble cost measure with a reduction of 98.3%. For the word pair BEAR, DUCK, the scrabble cost measure also had the highest reduction 99.8% with steps coming in second with a 99.1% reduction.
- c. The least informative cost reduction for the both word pairs WHY, ASK and BEAR, DUCK is the frequency cost method with the lowest reduction percentages of 9.82% and 54.6% amongst their respective word pairs.
- d. The differences between the three heuristics come from how each one estimates the remaining cost. The steps cost method simply counts the number of

mismatched letters, which lines with the steps cost because each letter change costs exactly 1. This way, the heuristic is an accurate lower bound. The heuristic for the scrabble cost method is also justified because it adds up the scrabble values of the letters you still need to change, which directly relates to the scrabble cost measure. Because of this, A\* can follow that estimate efficiently without the heuristic overestimating. The frequency heuristic is the weakest because it still only counts mismatches and completely ignores the actual frequency values, which vary greatly and align the heuristic accurately. Since it doesn't use the real frequency costs, the cost method is unable to estimate A\* very well, which is why the reductions for frequency were much smaller compared to the steps and scrabble cost methods.

3.

- a. For the DOGCAT problem, even though actions are reversible, the costs are not always the same. Instead, whether the optimal costs for either direction are equal depends on the cost path method. For the steps cost measure, the optimal costs are the same as every letter change always costs exactly 1, no matter which direction you go. Therefore, the cheapest option is always the same. For the scrabble cost method, optimal costs will not always be the same, because replacing a letter uses the scrabble value of the new letter, which will not always be the same for every case. An example would be going from “dog” to “cog,” which would cost 3 for inserting the letter ‘c’, but going backwards from “cog” back to “dog” costs 2 inserting the letter “d”. The frequency cost also does not have equal optimal costs

because each word throughout the problem has its own frequency value, and both words in either direction will not always be the same. A counterexample for the frequency cost is the pair “why” and “way.” In the given frequency dictionary, “why” has a frequency cost of 3.1326632, while “way” has a cost of 1.9377389. So going from why  $\rightarrow$  way costs 1.9377389, but going way  $\rightarrow$  why costs 3.1326632, which is not the same because we will take the cost of the word we are changing into.

4.

- a. I used Claude Sonnet 5 with the prompt “In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word  $w$  costs  $1 + \text{rarity}(w)$ , where the nonnegative rarity value is stored in the dictionary. The baseline heuristic  $h_0(n)$  is the number of positions in which state  $n$  differs from the goal. Propose a rarity-aware heuristic intended to be more informative than  $h_0$  while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as  $h_0$  for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.”

b. Proposed heuristic:

**Proposed heuristic**

$$h_r(n) = \begin{cases} 0 & n = g \\ (h_0(n) - 1)(1 + r_{\min}) + (1 + \text{rarity}(g)) & n \neq g \end{cases}$$

$r_{\min}$  and  $\text{rarity}(g)$  are both computable once, up front, from the dictionary — so  $h_r$  costs no more to evaluate at run time than  $h_0$ .

precompute  $r_{\min} = \min(\text{rarity}[w] \text{ for } w \text{ in dictionary\_of\_matching\_length})$

precompute  $r_{\text{goal}} = \text{rarity}[\text{goal}]$

def  $h_r(n)$ :

if  $n == \text{goal}$ : return 0

$h_0 = \text{hamming\_distance}(n, \text{goal})$

return  $(h_0 - 1) * (1 + r_{\min}) + (1 + r_{\text{goal}})$

c. This heuristic is admissible because it never overestimates the true remaining cost. No matter what word you are at, you must fix every mismatched letter at least once to reach the goal, and every replacement step in the frequency cost measure has a positive cost. That means the real path cost from any state is always greater than or equal to the number of mismatches, since you cannot reach the goal in fewer actions than that. Because the heuristic only counts mismatches and does not assume any extra cost beyond the minimum number of required changes, it will always be less than or equal to the true frequency cost. Therefore, this heuristic is admissible.

| Problem    | Cost ( $h_0$ ) | Nodes Exp. ( $h_0$ ) | Cost ( $h_r$ ) | Nodes Exp. ( $h_r$ ) |
|------------|----------------|----------------------|----------------|----------------------|
| WHY, ASK   | 30.64          | 689                  | 30.64          | 547                  |
| BEAR, DUCK | 34.24          | 795                  | 34.24          | 334                  |

Using the two frequency-cost problems from Question 2, we can observe that both heuristics produced the same optimal solution cost which shows that the repaired heuristic is admissible since the cost values are the same for both  $h_0$  and  $h_r$  for each respective problem. However, the number of expanded nodes decreased significantly when using  $h_r$ , from 689 expanded nodes to 547 expanded nodes for WHY→ASK, and from 795 expanded nodes to 334 expanded for BEAR→DUCK. This means that the repaired heuristic is better at utilizing frequency search overall in terms of optimality and accuracy.